Universidad Metropolitana

Departamento de Energía y Automatización

Robótica Industrial

PRÁCTICA Nº 5

PiCar-X con Python

Asignatura: Robótica Industrial (FPTEN27)

Profesor: José Montilva

Laboratorio de Robótica Industrial

Caracas, noviembre de 2026

Guía de Práctica de Laboratorio

Práctica #5 — PiCar-X con Python desde la Raspberry Pi

Objetivo General

Programar el robot móvil PiCar-X en Python directamente sobre la Raspberry Pi: ejecutar y leer los scripts de la librería `picarx`, modificar sus parámetros y escribir programas propios que combinen movimiento, sensores y visión.

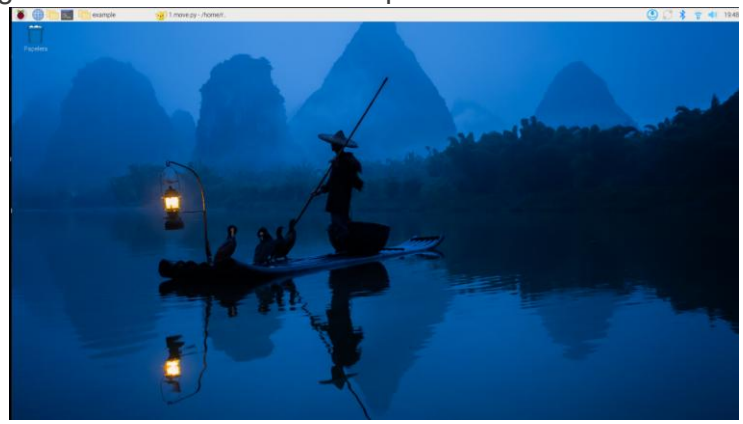
Antes de empezar — Conexión a la Raspberry Pi

⚠ Memoria SD correcta

- Esta práctica usa la memoria SD de **Python** (Raspberry Pi OS), no la de EzBlock.

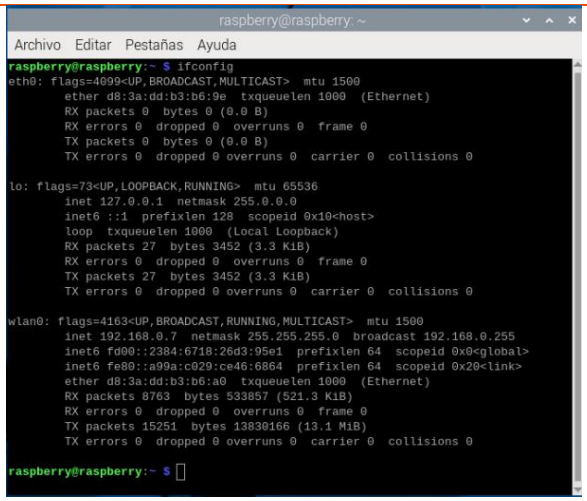
Parte A — Conexión inicial (con monitor)

1. Conecta un monitor a la Raspberry Pi por el puerto HDMI, y un teclado y un mouse por USB.
2. Enciende el carro con el interruptor del Robot HAT. Verifica que encienden las luces y se escucha el sonido de confirmación.
3. Espera a que cargue el escritorio del sistema operativo.



Escritorio de Raspberry Pi OS tras el arranque

4. Abre la terminal (ícono de pantalla negra, arriba a la izquierda) y escribe `ifconfig`.
5. Anota la dirección IP que aparece en la sección **wlan0** (por ejemplo, 192.168.0.7).



Resultado de ifconfig: la IP está en la sección wlan0

> **La IP puede cambiar**
>
> • Cada vez que el carro se conecta a la red WiFi puede recibir otra IP. Si pierdes la conexión, repite este paso.

Parte B — Acceso remoto desde tu laptop (VNC)

6. En tu laptop abre **VNC Viewer** e introduce la IP del paso 5.
7. Si aparece la advertencia de identidad del servidor, pulsa **Continuar**.
8. Inicia sesión con usuario `raspberry` y clave `Unimet`.



Barra de dirección de VNC Viewer con la IP del PiCar-X

Con la sesión VNC abierta ya puedes desconectar monitor, teclado y mouse: a partir de ahora todo se controla desde la laptop.

Cómo ejecutar y editar los scripts

Los ejemplos están en la carpeta `~/picar-x/example`. Para ejecutarlos, abre la terminal:

```
cd ~/picar-x/example
sudo python3 1.move.py
```

Para detener cualquier programa pulsa **Ctrl+C**. Para leer o editar un script ábrelo en el IDE **Thonny** (doble clic desde el gestor de archivos).

> **Trabaja sobre copias**
>
> • Antes de modificar un ejemplo, haz una copia con el nombre de tu grupo y edita solo la copia:
> `cp 4.avoiding_obstacles.py g3_avoiding.py`
> • Los archivos originales los necesita el siguiente grupo.
> • Usa `sudo` al ejecutar: sin él, el audio y algunos periféricos no funcionan.

⚠ Seguridad

- Trabaja en el piso o con espacio libre alrededor del carro; nunca lo pongas a andar sobre una mesa.
- Mantén la terminal enfocada para poder detenerlo con **Ctrl+C**, o ten la mano cerca del interruptor del Robot HAT.

Ejercicio #1 Movimiento y teleoperación

Todo el control del carro pasa por un objeto de la clase `Picarx`. Estas son las funciones de movimiento que usarás durante la práctica:

Función	Qué hace	Rango
`px.forward(v)`	Avanza con potencia v	0 a 100
`px.backward(v)`	Retrocede con potencia v	0 a 100
`px.stop()`	Detiene los motores	—
`px.set_dir_servo_angle(a)`	Gira las ruedas delanteras	−30° a 30°
`px.set_cam_pan_angle(a)`	Mueve la cámara a izquierda/derecha	−90° a 90°
`px.set_cam_tilt_angle(a)`	Mueve la cámara arriba/abajo	−35° a 65°

Paso 1 — 1.move.py

Ejecútalo: el carro avanza haciendo un giro en S, se detiene y mueve la cámara en horizontal y en vertical. Ábrelo en Thonny y ubica cada bloque `for` que produce esos movimientos.

Paso 2 — 2.keyboard_control.py

Conduce el carro con las teclas **W, A, S, D** (Ctrl+C para salir). Observa la latencia entre la pulsación y la respuesta: los comandos viajan de tu laptop a la Raspberry Pi por WiFi a través de VNC.

★ Reto — Escribe tu propia trayectoria

Crea un archivo nuevo `mi_trayectoria.py` y completa los tiempos para que el carro recorra un cuadrado de unos 50 cm de lado:

```
from picarx import Picarx
import time

px = Picarx()
try:
    for i in range(4):
        px.set_dir_servo_angle(0)      # ruedas rectas
        px.forward(30)
        time.sleep(____)               # tramo recto
        px.set_dir_servo_angle(30)     # girar
        px.forward(30)
        time.sleep(____)               # giro de ~90 grados
finally:
    px.stop()                          # se ejecuta aunque pulses Ctrl+C
    px.set_dir_servo_angle(0)
```

Luego conviértelo en una función `cuadrado(lado_s, giro_s)` y úsala para recorrer el cuadrado dos veces seguidas. Mide cuánto queda el carro del punto de partida.

⚠ ¿Qué observar?

- ¿Para qué sirve el bloque `try / finally`? ¿Qué pasaría sin él si detienes el programa con Ctrl+C a mitad de un tramo?
- Al igual que con bloques, el carro se mueve por tiempo (lazo abierto): ¿de qué depende que el cuadrado cierre?

Ejercicio #2 Sensores y comportamiento reactivo

Un comportamiento **reactivo** repite sin parar el ciclo: **leer sensores** → **comparar con umbrales** → **actuar**. En Python ese ciclo es un `while True`.

Paso 1 — 4.avoiding_obstacles.py

Ejecútalo y coloca objetos frente al carro. Luego ábrelo en Thonny: el programa usa la lectura `px.ultrasonic.read()` (en cm) y dos umbrales que definen tres zonas.

```
POWER = 50
SafeDistance = 40     # > 40 cm: zona segura, avanza recto
DangerDistance = 20   # 20 a 40 cm: gira para esquivar
                      # < 20 cm: retrocede
```

En tu copia del script prueba las configuraciones de la tabla y describe el comportamiento.

Configuración	POWER	SafeDistance	DangerDistance	Comportamiento observado
A (original)	50	40	20	
B	30	60	30	
C	50	25	10	

Registra estos valores en la bitácora del grupo.

Paso 2 — 6.cliff_detection.py

Este script usa el sensor de escala de grises para detectar un "precipicio" y retrocede bruscamente (`px.backward(80)`) diciendo "danger". Pruébalo en el piso acercando el carro a una hoja de papel negro. **No lo pruebes en el borde de una mesa.**

> **★ Reto — Monitor de sensores**
>
> Escribe `monitor.py`: con el carro **detenido**, imprime cada 0,2 s la distancia del ultrasonido, los 3 valores de escala de grises (`px.get_grayscale_data()`) y la zona en la que está el obstáculo ("SEGURA", "PRECAUCIÓN" o "PELIGRO").
>
> Además, cuando el obstáculo **entra** a la zona de peligro, el carro debe decir "danger" una sola vez (no en cada lectura). Pista: guarda la zona anterior en una variable, como hace `6.cliff_detection.py` con `last_state`.

```
from picarx import Picarx
from robot_hat import TTS
import time

px = Picarx()
tts = TTS()
tts.lang("en-US")

while True:
    d = round(px.ultrasonic.read(), 1)
    gris = px.get_grayscale_data()
    # completa: decidir la zona y avisar al entrar en PELIGRO
    print(d, gris)
    time.sleep(0.2)
```

> **⚠ ¿Qué observar?**
>
> • ¿Las lecturas del ultrasonido son estables con el carro quieto? ¿Por qué podrían aparecer valores sueltos fuera de lugar?
>
> • ¿Qué cambia en el comportamiento del carro al acercar los dos umbrales entre sí?

Ejercicio #3 Seguimiento de línea

Arma con cinta aislante negra un circuito cerrado sobre una superficie clara, con al menos una curva abierta y una cerrada.

Paso 1 — Calibrar el sensor de escala de grises

Los umbrales que separan "línea" de "fondo" dependen de la cinta, del piso y de la luz del salón, así que primero se calibran:

```
cd ~/picar-x/example/calibration
sudo python3 grayscale_calibration.py
```

1. Pulsa **Q** para calibrar la referencia de línea.
2. Desliza el carro a mano para que los tres sensores pasen varias veces sobre la cinta y sobre el fondo.

3. Pulsa **ESPACIO** para guardar la calibración y **Ctrl+C** para salir.

Paso 2 — 5.minecart_plus.py

Ejecútalo sobre el circuito. Luego ábrelo en Thonny y estudia la función `get_status()`: traduce las tres lecturas en uno de cuatro estados — `forward`, `left`, `right` o `stop` — y la función `outHandle()` intenta recuperar la línea cuando el carro la pierde. Dibuja en la bitácora el diagrama de estados del programa.

Paso 3 — Ajusta los parámetros

En tu copia cambia `px_power` (velocidad) y `offset` (ángulo de corrección de la dirección).

px_power	offset	¿Completa la vuelta?	Tiempo de vuelta (s)	Veces que pierde la línea
10	20			
20	20			
20	30			

Registra estos valores en la bitácora del grupo.

★ Reto — Línea con frenado de emergencia

En tu copia de `5.minecart_plus.py` agrega el ultrasonido dentro del ciclo principal: si hay un obstáculo a **menos de 20 cm**, el carro se detiene (`px.stop()`) y no sigue la línea hasta que el obstáculo desaparezca.

⚠ ¿Qué observar?

- ¿Por qué una velocidad alta hace que el carro pierda la línea en las curvas cerradas?
- ¿Qué hace `offset`? ¿Qué pasa si es muy pequeño o muy grande?

Ejercicio #4 Visión y reto integrador

La librería **Vilib** de SunFounder usa OpenCV para procesar la imagen de la cámara y guarda los resultados en el diccionario `Vilib.detect_obj_parameter`: cuántos objetos detectó y en qué coordenadas (x, y) de la imagen de 640 × 480 píxeles.

Paso 1 — 7.display.py

Ejecútalo y usa el teclado: **1 a 6** activan la detección de rojo, naranja, amarillo, verde, azul y púrpura; **0** la apaga; **f** activa la detección de rostros; **s** imprime la posición y el tamaño del objeto detectado. Prueba con objetos del laboratorio.

Paso 2 — 8.stare_at_you.py

La cámara sigue tu rostro. Ábrelo y analiza esta línea:

```
x_angle += (coordinate_x*10/640) - 5
```

Si el rostro está en el centro (x = 320) la corrección es 0; mientras más lejos del centro, mayor es la corrección. Es un **control proporcional**: la acción es proporcional al error.

Paso 3 — 10.bull_fight.py

El carro persigue un objeto rojo usando la misma idea: orienta la cámara hacia el color y gira las ruedas hacia ese ángulo. Mueve el objeto lentamente y observa la respuesta.

★ Reto — Guardia de seguridad autónomo

Escribe desde cero `guardia.py`. El carro patrulla un área y debe cumplir **todos** estos requisitos:

Req.	Descripción
R1	**Movimiento:** avanza en línea recta a velocidad moderada (entre 20 y 40).
R2	**Detección:** lee continuamente el ultrasonido; si hay un obstáculo a menos de 25 cm, se detiene.
R3	**Alerta:** al detenerse dice "Obstacle detected" con TTS.
R4	**Escaneo:** detenido, mueve la cámara de izquierda a derecha (pan) para "inspeccionar".
R5	**Evasión:** retrocede un poco, gira para esquivar el obstáculo y reanuda la patrulla.
R6	**Bucle:** todo ocurre dentro de un `while True` y el programa termina con Ctrl+C dejando el carro detenido.

Funciones de referencia: `px.forward(v)`, `px.backward(v)`, `px.stop()`, `px.set_dir_servo_angle(a)`, `px.set_cam_pan_angle(a)`, `px.ultrasonic.read()`, `tts.say("texto")`.

⚠ ¿Qué observar?

- ¿Por qué stare_at_you.py suma la corrección a `x_angle` en lugar de asignar directamente un ángulo?
- En el guardia, ¿qué pasa si el obstáculo sigue ahí después de la evasión? ¿Cómo lo resolvieron?

Al finalizar — Apagado

1. Apaga la Raspberry Pi desde el menú del sistema: ícono de la frambuesa → **Salir** → **Shutdown**.
2. Cuando la Raspberry Pi termine de apagarse, apaga el carro con el interruptor del Robot HAT.

⚠ Primero el sistema, luego el interruptor

- Apagar el carro sin cerrar el sistema operativo puede corromper la memoria SD.